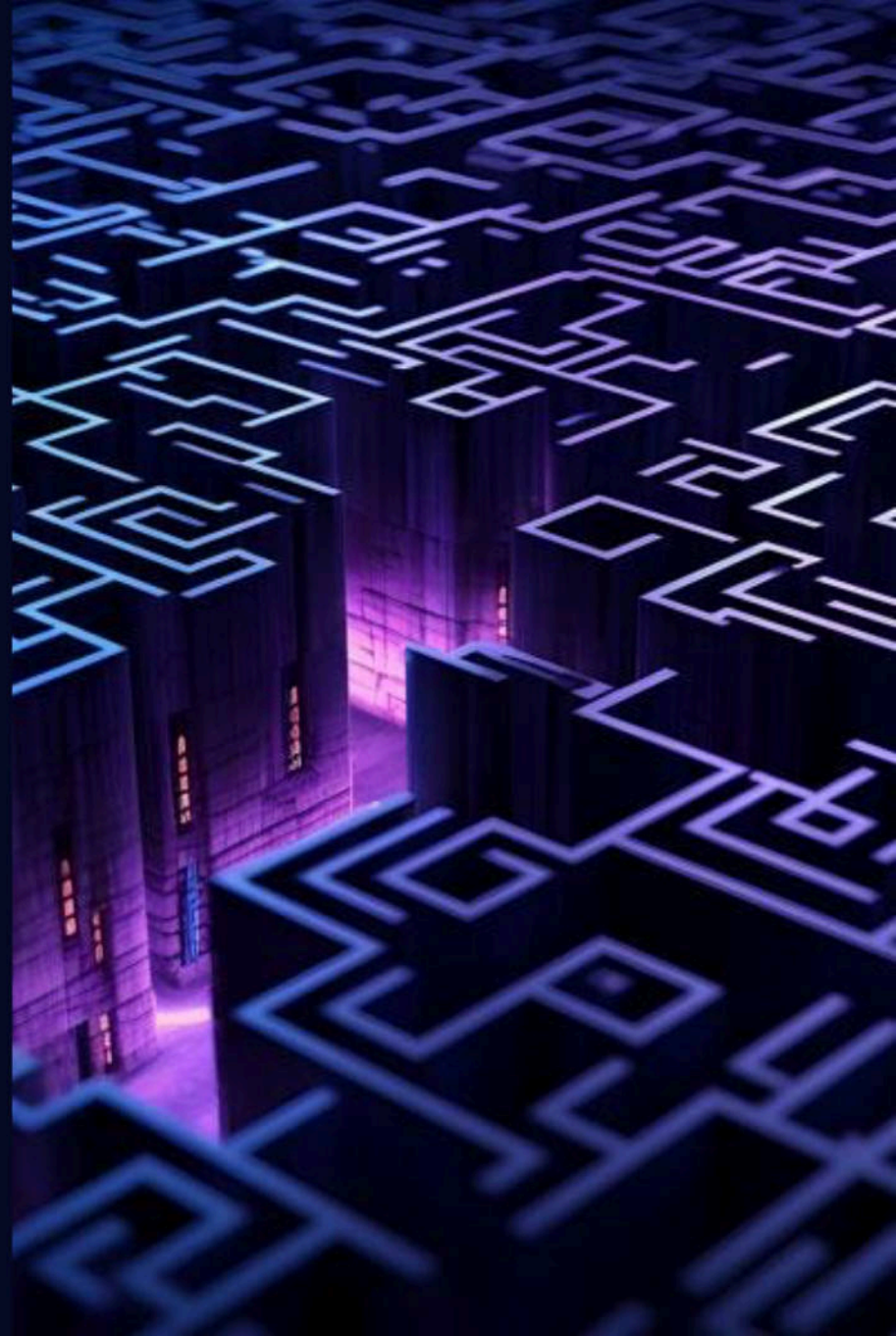




ICONV, set the charset to RCE

EXPLOITING THE GLIBC TO HACK THE PHP ENGINE

Charles Fol
@cfreal_



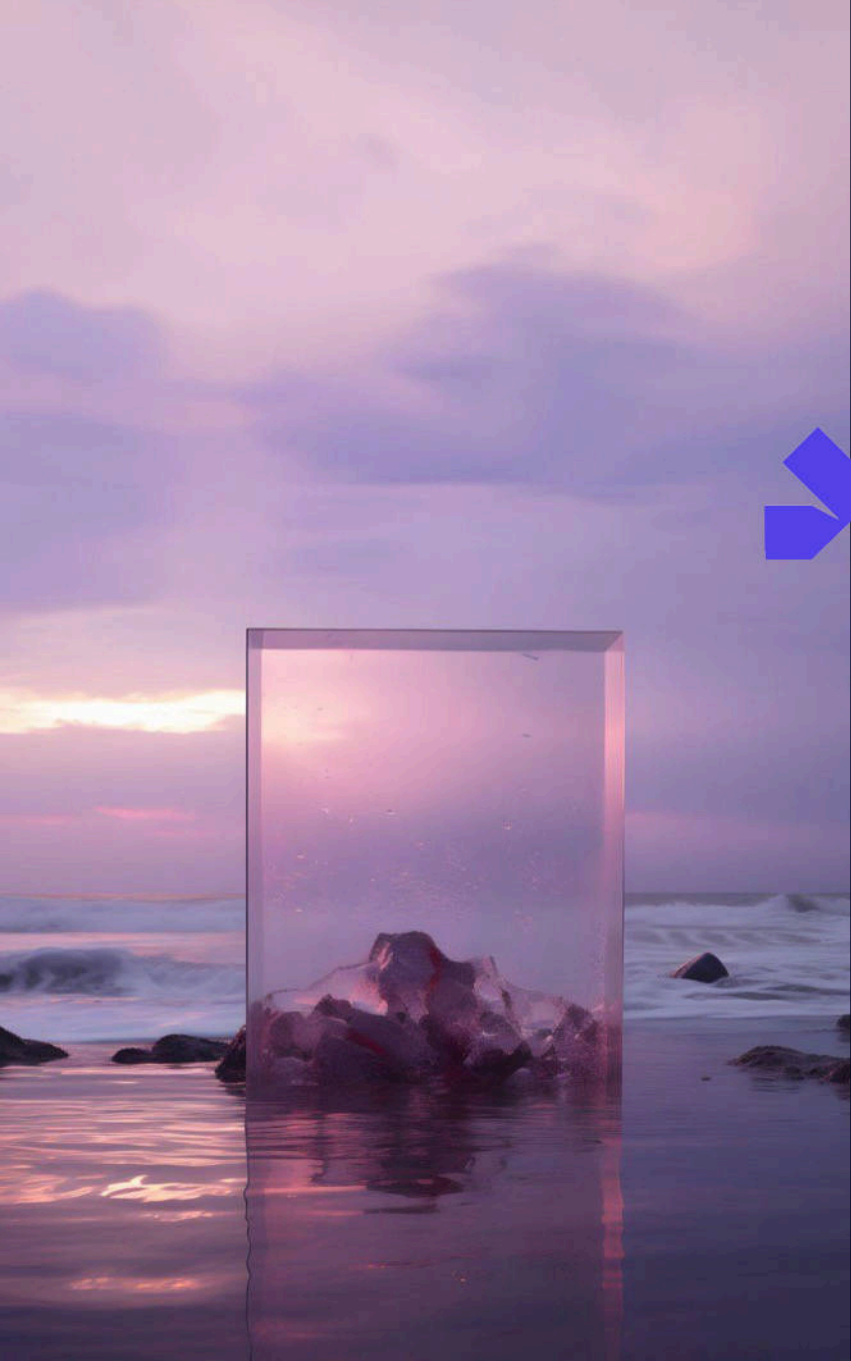
Summary

Discovery of CVE-2024-2961

Attacking PHP filters

Attacking direct iconv() calls

Conclusion



Discovery

PHP file read primitive

What can you do with a file read primitive?

```
echo file_get_contents($_GET['file']);
```



PHP file read primitive

What can you do with a file read primitive?

```
echo file_get_contents($_GET['file']);
```

```
echo file_get_contents('/etc/passwd');
```



PHP file read primitive

What can you do with a file read primitive?

```
echo file_get_contents($_GET['file']);
```

```
echo file_get_contents('/etc/passwd');
```

```
echo file_get_contents('http://ambionics.io/blog');
```



PHP file read primitive

What can you do with a file read primitive?

```
echo file_get_contents($_GET['file']);
```

```
echo file_get_contents('/etc/passwd');
```

```
echo file_get_contents('http://ambionics.io/blog');
```

```
echo file_get_contents('ftp://ftp.ambionics.io/file.bin');
```



PHAR : a dangerous protocol

```
phar:///var/www/archive.phar/file.txt
```

PHAR: PHP archive

- Source files
- Resources
- Serialized metadata
 - Deserialized when reading an archive



PHAR : File read to RCE

Harder to exploit

- PHAR:
 - PHP 8+: no more **deserialisation** of the metadata
 - **phar://** might be **disabled** (Drupal, Magento)
- Deserialisation:
 - Gadget chains are being **patched** more and more
 - **Types** are making a comeback



PHP filters: applying transforms to the stream

```
php://filter/[filters]/resource=[resource]
```

[resource]
[filters]

original stream: file, etc.

list of filters to apply to the stream



PHP filters: applying transforms to the stream

php://filter/convert.base64-encode/resource=/etc/passwd

cm9vdDp40jA6MDpyb2900i9yb2900i9iaW4vYXNoCmJpbj40jE6MTpiaW46L2Jpbjovc2Jpb9ub2xvZ2luCmRhZW1vbJp40jI6MjpkYWVtb246L3NiaW46L3NiaW4vbm9sb2dpbgphZG06eDoz0jQ6YWRt0i92YXIVYWRt0i9zYmluL25vbG9naW4KbHA6eDo0jc6bHA6L3Zhci9zcG9vbC9scGQ6L3NiaW4vbm9sb2dpbgpzeW5jOng6NTowOnN5bmM6L3NiaW46L2Jpb9zeW5jCnNodXRkb3duOng6NjowOnNodXRkb3du0i9zYmlu0i9zYmluL3NodXRkb3duCmhhbHQ6eDo30jA6aGFsdDovc2Jpbjovc2Jpb9oYWw0Cm1haWw6eDo40jEyOm1haWw6L3Zhci9tYWls0i9zYmluL25vbG9naW4KbmV3czp40jk6MTM6bmV3czovdXNyL2xpYi9uZXdz0i9zYmluL25vbG9naW4KdXVjcDp40jEw0jE00nV1Y3A6L3Zhci9zcG9vbC91dWNwHVibGlj0i9zYmluL25vbG9naW4Kb3BlcmF0b3I6eDoxMTowOm9wZXJhdG9y0i9yb2900i9zYmluL25vbG9naW4KbWFWOng6MTM6MTU6bWFWu0i91c3IvbwFu0i9zYmluL25vbG9naW4KcG9zdGh1c3Rlcj40jE00jEy0nBvc3RtYXN0ZXI6L3Zhci9tYWls0i9zYmluL25vbG9naW4KY3JvbJp40jE20jE20mNyB246L3Zhci9zcG9vbC9jcm9u0i9zYmluL25vbG9naW4KZnRwOng6MjE6MjE60i92YXIVbGliL2Z0cDovc2Jpb9ub2xvZ2luCnNzAQG6eDoyMjoyMjpc2hk0i9kZXVbnVsbDovc2Jpb9ub2xvZ2luCmF0Ong6MjU6MjU6YXQ6L3Zhci9zcG9vbC9jcm9uL2F0am9iczovc2Jpb9ub2xvZ2luCnNxdWlkOng6MzE6MzE6U3F1aWQ6L3Zhci9jYWN0ZS9zcXVpZDovc2Jpb9ub2xvZ2luCnhmczp40jMz0jMz0lggRm9udCBTZXJ2ZXI6L2V0Yy9YMTevZnM6L3NiaW4vbm9sb2dpbgpnYW1lczp40jM10jM10mdhbWVz0i91c3IvZ2FtZXM6L3NiaW4vbm9sb2dpbgpjexXJ1czp40jg10jEy0jovdXNyL2N5cnVz0i9zYmluL25vbG9naW4KdnBvcG1haWw6eDo40t040t06L3Zhci9zcG9wbWpfbDovc2Jpb9ub2xvZ2luCm50cDp40jEYmzoxMjM6TLRQ0i92YXIVZW1wdHk6L3NiaW4vbm9sb2dpbgpzbW1zcDp40jIw0ToymDk6c21tcA6L3Zhci9zcG9vbC9tXCVldWU6L3NiaW4vbm9sb2dpbgpndWVzdDp40jQwNTowMDA6Z3Vlc3Q6L2Rldi9udWxs0i9zYmluL25vbG9naW4Kbm9ib2R5Ong6NjU1MzQ6NjU1MzQ6bm9ib2R50i86L3NiaW4vbm9sb2dpbgpuZ2luZDp40jEwMDoxMDE6bmdpbng6L3Zhci9saWVbmdpbng6L3NiaW4vbm9sb2dpbgp2bnN0YXQ6eDoxMDE6MTAyOnZuc3RhDOvdmFyL2xpYi92bnN0YXQ6L2Jpb9mYWxzZQpyZWRpczp40jEwMjoxMDM6cmVkaXM6L3Zhci9saWVcmVkaXM6L2Jpb9mYWxzZQ==



PHP filters: applying transforms to the stream

php://filter/**convert.base64-encode**|**convert.base64-encode**/resource= ...

```
Y205dmREcDRPakE2TURweWIyOTBPaTl5YjI5ME9pOWlhVzR2VWVhOb0NtSnBianA0T2pFNk1UcGlhVzQ2
TDJKcGJqb3ZjMkpWYmk5dWIyeHZaMmx1Q21SaFpXMXZianA0T2pJNk1qcGtZV1Z0YjI0NkwzTmlhVzQ2
TDN0aWFXNHZ1bTlZyYjKcGJncGhRzA2ZURvek9qUTZ2V1J0T2k5MllySXZZV1J0T2k5elltbHVMMjV2
Ykc5bmFXNEtiSEe2ZURvME9qYzZiSEe2TDNaaGNpOXpJRzL2YkM5c2NHUTZMM05pYVc0dmJtOXNiMmRw
YmdwemVXNWpPbmc2TlRvd09uTjVibU02TDN0aWFXNDZMMkpWYmk5emVXNWpDbk5vZFHsa2IzZHVpBmc2
TmPvd09uTm9kWFJrYjNkdU9pOXpZbWx1T2k5elltbHVMM05vZFHsa2IzZHVDbWhoYkhrNmVEbzNPakE2
YUdGc2REb3ZjMkpWYmpvdmMySnBiaTlVwVd4MENTMWhhV3c2ZURvNE9qRXLPbTFoYVd3NkwzWmhjaTl0
WVdsc09pOXpZbWx1TDI1dmJH0W5hVzRlYm1WM2N6cDRPams2TVRNNmJtVjNjem92ZFh0eUwyeHBZaTl1
Wlhkek9pOXpZbWx1TDI1dmJH0W5hVzRlZFHwamNEcDRPakV3T2pFME9uVjFZM0E2TDNaaGNpOXpJRzL2
YkM5MWRXTndjSFZpYkdsak9pOXpZbWx1TDI1dmJH0W5hVzRlYjNCbGNtRjBiM0k2ZURveE1Ub3dPbTl3
WlhKaGRHOXLPaTl5YjI5ME9pOXpZbWx1TDI1dmJH0W5hVzRlYlGdU9uZzZNVe02TVRVNmJXRnVPaTkx
YzNjdmJXRnVPaTl6WW1sdUwyNXZiRzluYVc0S2NH0XpkRzFoYzNSbGNgcDRPakUwT2pFeU9uQnZjM1J0
WVh0MFpYSTZMM1poY2k5dFlXbHNPATl6WW1sdUwyNXZiRzluYVc0S1kzSnZianA0T2pFMk9qRTJPbU55
YjI0NkwzWmhjaTl6Y0c5dmJDOWpjbTl1T2k5elltbHVMMjV2Ykc5bmFXNEtablJ3T25nNk1qRTZNakU2
T2k5MllySXZiR2xpTDJaMGNEb3ZjMkpWYmk5dWIyeHZaMmx1Q250emFHUTZlRG95TWpveU1qcHjJmMhr
T2k5a1pYWXZibLZzYkRvdmMySnBiaTl1YjJ4dloybHVDdbUYwT25nNk1qVTZNaLU2WVhRNkwzWmhjaTl6
Y0c5dmJDOWpjbTl1TDJGMGftOWlJem92YzJKcGJpOXViMnh2WjJsdUNuTnhkV2xrT25nNk16RTZNekU2
VTNGMWFxUTZMM1poY2k5allXTm9aUzL6Y1hWcFpEb3ZjMkpWYmk5dWIyeHZaMmx1Q250bWN6cDRPak16
T2pNek9sZ2dSbTl1ZENCVFpYSjJaWEk2TDJWMFL5OVlNVEV2Wm5NNkwzTmlhVzR2Ym05c2IyZHBiZ3Bu
WVcxbsGN6cDRPak0xT2pNMU9tZGhiV1Z6T2k5MMWzSXZaMkZ0WlhNNkwzTmlhVzR2Ym05c2IyZHBiZ3Bq
ZVhKMWN6cDRPamcxT2pFeU9qb3ZkWE55TDJONWNUvnpPaTl6WW1sdUwyNXZiRzluYVc0S2RuQnZjRzFo
YVd3NmVEbzRPVG80T1RvNkwzWmhjaTkyY0c5d2JXRnBiRG92YzJKcGJpOXViMnh2WjJsdUNTNTBjRHA0
T2pFeU16b3hNak02VGxSUU9pOTJZWEL2WlCxd2RIazZMM05pYVc0dmJtOXNiMmRwYmdwemJXMXpjRHA0
T2pJd09Ub3lNRGs2YzIxdGMzQTZMM1poY2k5emNH0XZiQzL0Y1hwbGRXVTZMM05pYVc0dmJtOXNiMmRw
YmdwbmRXVnpkRHA0T2pRd05Ub3hNREU2Ym1kcGJuzZMM1poY2k5c2FXSXZibWRwYm5nNkwzTmlhVzR2
Ym05c2IyZHBiZ3AyYm50MFLYUTZlRG94TURFNk1UQXLPbLp1YzNSaGREb3ZkbUZ5TDJ4cFlpOTJibk4w
WVhRNkwYnBiaTlVwVd4elpRcHlaV1JwY3pwNE9qRXdNam94TURNmNtVmtHWE02TDNaaGNpOXNhV0L2
Y21Wa2FYTTZMMkpWYmk5bVlXhEpaUT09
```



PHP filters: lots of options

Many available filters

- `string.upper` *Convert stream to uppercase*
- `string.lower` *Convert stream to lowercase*
- `string.rot13` *Do some BC crypto*
- `dechunk` *Decode chunked encoding*
- `convert.iconv.X.Y` *Convert from charset `X` to charset `Y`*



PHP filters: applying transforms to the stream

php://filter/convert.iconv.UTF-8.UTF-16/resource=/etc/passwd

```
00000000: fffe 7200 6f00 6f00 7400 3a00 7800 3a00 ..r.o.o.t.:x:.
00000010: 3000 3a00 3000 3a00 7200 6f00 6f00 7400 0.:0.:r.o.o.t.
00000020: 3a00 2f00 7200 6f00 6f00 7400 3a00 2f00 :./r.o.o.t:./
00000030: 6200 6900 6e00 2f00 6100 7300 6800 0a00 b.i.n./a.s.h..
00000040: 6200 6900 6e00 3a00 7800 3a00 3100 3a00 b.i.n.:x:.1:.
00000050: 3100 3a00 6200 6900 6e00 3a00 2f00 6200 1.:b.i.n:./b.
00000060: 6900 6e00 3a00 2f00 7300 6200 6900 6e00 i.n:./s.b.i.n.
00000070: 2f00 6e00 6f00 6c00 6f00 6700 6900 6e00 /n.o.l.o.g.i.n.
00000080: 0a00 6400 6100 6500 6d00 6f00 6e00 3a00 ..d.a.e.m.o.n:.
00000090: 7800 3a00 3200 3a00 3200 3a00 6400 6100 x.:2.:2.:d.a.
000000a0: 6500 6d00 6f00 6e00 3a00 2f00 7300 6200 e.m.o.n:./s.b.
000000b0: 6900 6e00 3a00 2f00 7300 6200 6900 6e00 i.n:./s.b.i.n.
000000c0: 2f00 6e00 6f00 6c00 6f00 6700 6900 6e00 /n.o.l.o.g.i.n.
000000d0: 0a00 6100 6400 6d00 3a00 7800 3a00 3300 ..a.d.m.:x:.3.
000000e0: 3a00 3400 3a00 6100 6400 6d00 3a00 2f00 :.4.:a.d.m:./
000000f0: 7600 6100 7200 2f00 6100 6400 6d00 3a00 v.a.r./a.d.m:.
00000100: 2f00 7300 6200 6900 6e00 2f00 6e00 6f00 /s.b.i.n./n.o.
00000110: 6c00 6f00 6700 6900 6e00 0a00 6c00 7000 l.o.g.i.n..l.p.
00000120: 3a00 7800 3a00 3400 3a00 3700 3a00 6c00 :x.:4.:7.:l.
00000130: 7000 3a00 2f00 7600 6100 7200 2f00 7300 p:./v.a.r./s.
00000140: 7000 6f00 6f00 6c00 2f00 6c00 7000 6400 p.o.o.l./l.p.d.
```



PHP filters: arbitrary suffix

Arbitrary **prefix** and **suffix**? ^[1]

```
$json = file_get_contents($_GET['file']);  
$json = json_decode($json);  
  
echo $json['message']['title'];
```

[1] <https://www.ambionics.io/blog/wrapwrap-php-filters-suffix>



PHP filters: finding the bug

Bruteforcing conversions

```
php://filter/convert.iconv.X.Y/resource=data:,test123
```

```
segmentation fault (core dumped)
```



A crash

Woops, a crash

`iconv`: glibc's character set conversion library

```
iconv_t iconv_open(const char *tocode, const char *fromcode);
size_t iconv(iconv_t cd,
              char **inbuf, size_t *inbytesleft,
              char **outbuf, size_t *outbytesleft);
```

Processes at most **inbytesleft** bytes from **inbuf** and writes at most **outbytesleft** bytes to **outbuf***

*in theory



The vulnerability: OOB write

```
glibc-2.39/iconvdata/iso-2022-cn-ext.c

1 if ((used & SO_mask) != 0 && (ann & SO_ann) != (used << 8))
2 {
3     if (outptr + 4 > outend)
4     {
5         result = __GCONV_FULL_OUTPUT;
6         break;
7     }
8
9     assert(used >= 1 && used <= 4);
10    escseq = ")A\0\0)G)E" + (used - 1) * 2;
11    *outptr++ = ESC;
12    *outptr++ = '$';
13    *outptr++ = *escseq++;
14    *outptr++ = *escseq++;
15
16    ann = (ann & ~SO_ann) | (used << 8);
17 }
18 else if ((used & SS2_mask) != 0 && (ann & SS2_ann) != (used << 8))
19 {
20     assert(used == CNS11643_2_set); /* XXX */
21     escseq = "*H";
22     *outptr++ = ESC;
23     *outptr++ = '$';
24     *outptr++ = *escseq++;
25     *outptr++ = *escseq++;
26
27     ann = (ann & ~SS2_ann) | (used << 8);
28 }
29 else if ((used & SS3_mask) != 0 && (ann & SS3_ann) != (used << 8))
30 {
31     assert((used >> 5) >= 3 && (used >> 5) <= 7);
32     escseq = "+I+J+K+L+M" + ((used >> 5) - 3) * 2;
33     *outptr++ = ESC;
34     *outptr++ = '$';
35     *outptr++ = *escseq++;
36     *outptr++ = *escseq++;
37
38     ann = (ann & ~SS3_ann) | (used << 8);
39 }
```

When converting to **ISO-2022-CN-EXT**, the library fails to check the bounds before writing escape sequences to the output buffer

Overflow of 1 to 3 bytes:

\$*H	24	2A	48
\$+I	24	2B	49
\$+J	24	2B	4A
\$+K	24	2B	4B
\$+L	24	2B	4C
\$+M	24	2B	4D



The vulnerability: OOB write

```
glibc-2.39/iconvdata/iso-2022-cn-ext.c

1 if ((used & SO_mask) != 0 && (ann & SO_ann) != (used << 8))
2 {
3     if (outptr + 4 > outend)
4     {
5         result = __GCONV_FULL_OUTPUT;
6         break;
7     }
8
9     assert(used >= 1 && used <= 4);
10    escseq = "A\0\0G)E" + (used - 1) * 2;
11    *outptr++ = ESC;
12    *outptr++ = '$';
13    *outptr++ = *escseq++;
14    *outptr++ = *escseq++;
15
16    ann = (ann & ~SO_ann) | (used << 8);
17 }
18 else if ((used & SS2_mask) != 0 && (ann & SS2_ann) != (used << 8))
19 {
20     assert(used == CNS11643_2_set); /* XXX */
21     escseq = "*H";
22     *outptr++ = ESC;
23     *outptr++ = '$';
24     *outptr++ = *escseq++;
25     *outptr++ = *escseq++;
26
27     ann = (ann & ~SS2_ann) | (used << 8);
28 }
29 else if ((used & SS3_mask) != 0 && (ann & SS3_ann) != (used << 8))
30 {
31     assert((used >> 5) >= 3 && (used >> 5) <= 7);
32     escseq = "+I+J+K+L+M" + ((used >> 5) - 3) * 2;
33     *outptr++ = ESC;
34     *outptr++ = '$';
35     *outptr++ = *escseq++;
36     *outptr++ = *escseq++;
37
38     ann = (ann & ~SS3_ann) | (used << 8);
39 }
```

CVE-2024-2961

AAAAAA 割
BBBBBB 破
CCCCCC 刈
DDDDDD 湿



Tedious conditions

Vulnerability

- 1, 2 or 3 bytes overflow
- **fixed** byte values

Conditions

- Control the output charset ([ISO-2022-CN-EXT](#))
- Control part of the input buffer
- Have a suitable output buffer





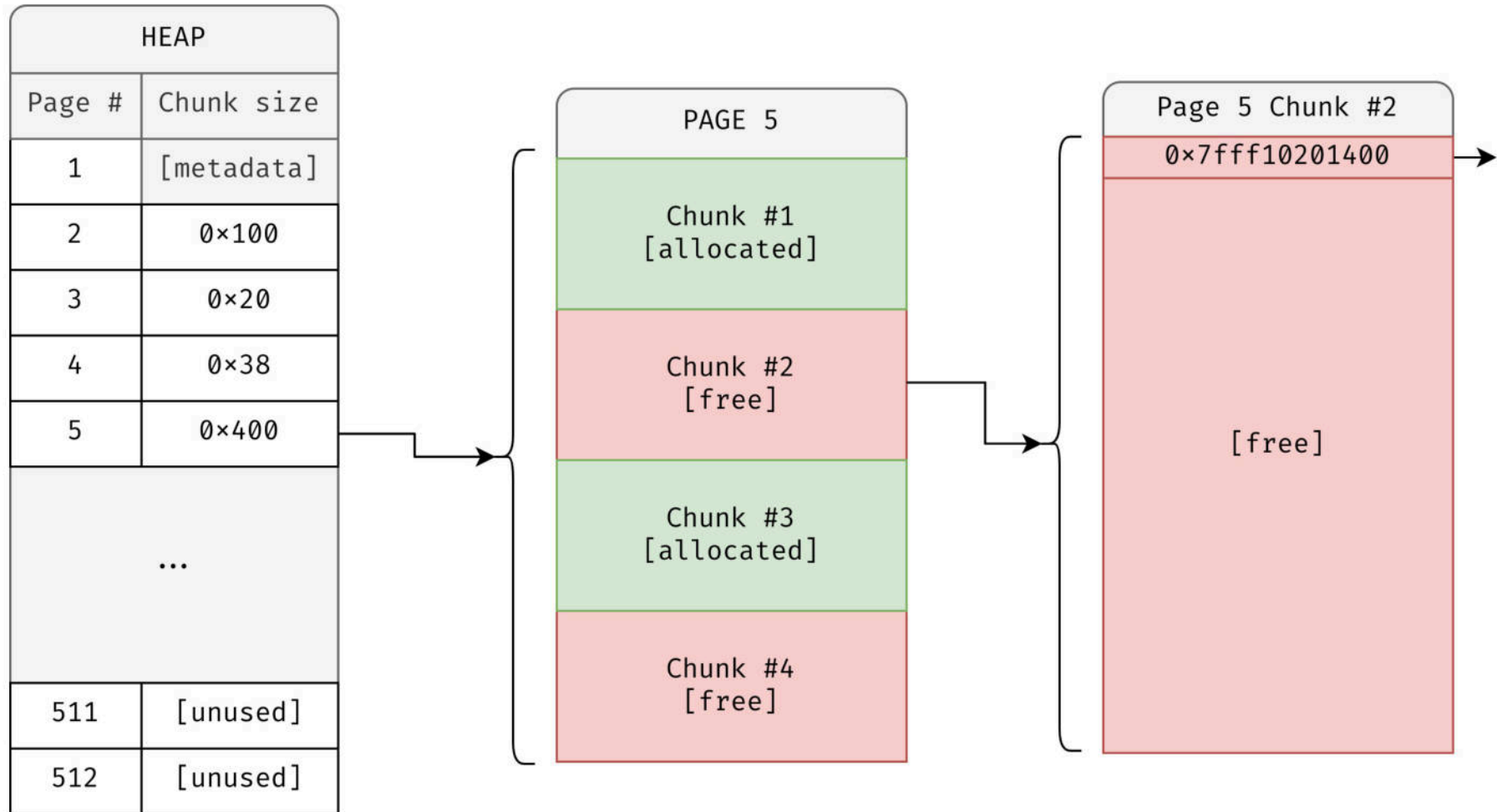
Attacking PHP filters

PHP's heap

- PHP's heap is **page-based**
 - 512 pages of 0x1000 bytes
- Each page contains **chunks** of the same size
 - Page 10: chunks of size 0x100
 - Page 11: chunks of size 0x30
 - Page 12: chunks of size 0xa00
- PHP holds a singly-linked **free list** for each chunk size
 - Last In, First Out (LIFO)
 - Similar to libc's tcache, but unlimited

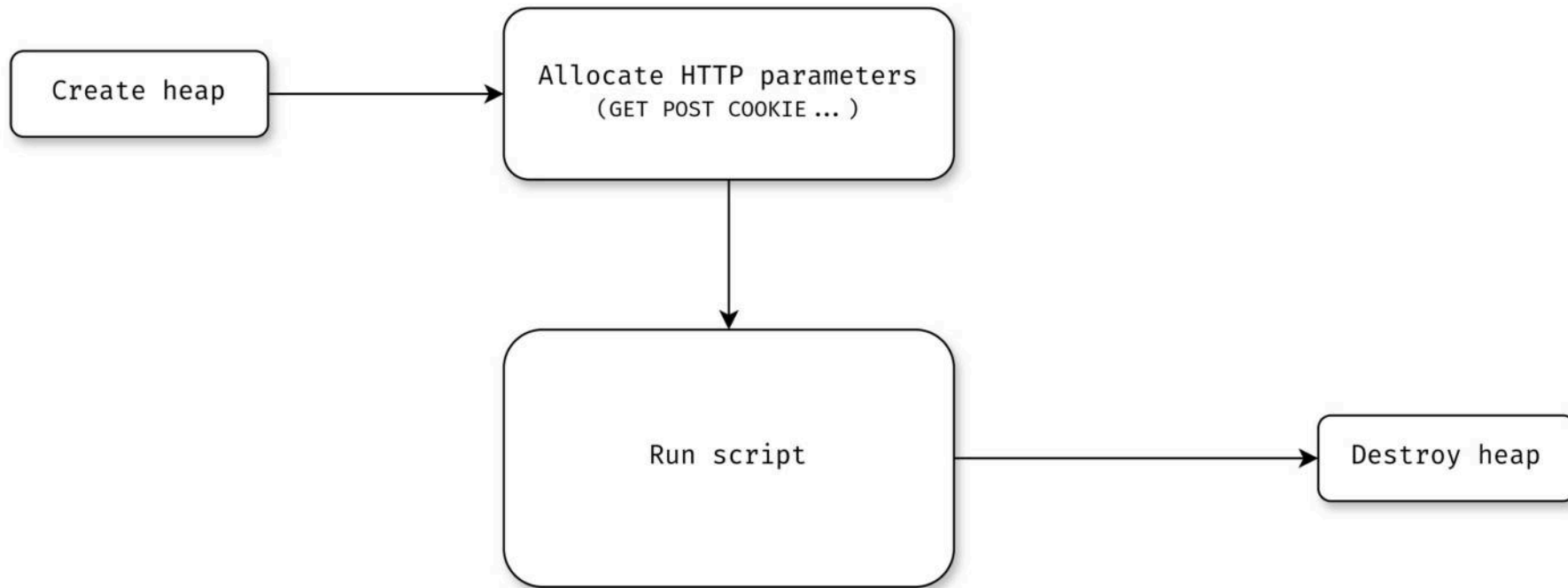


PHP's heap



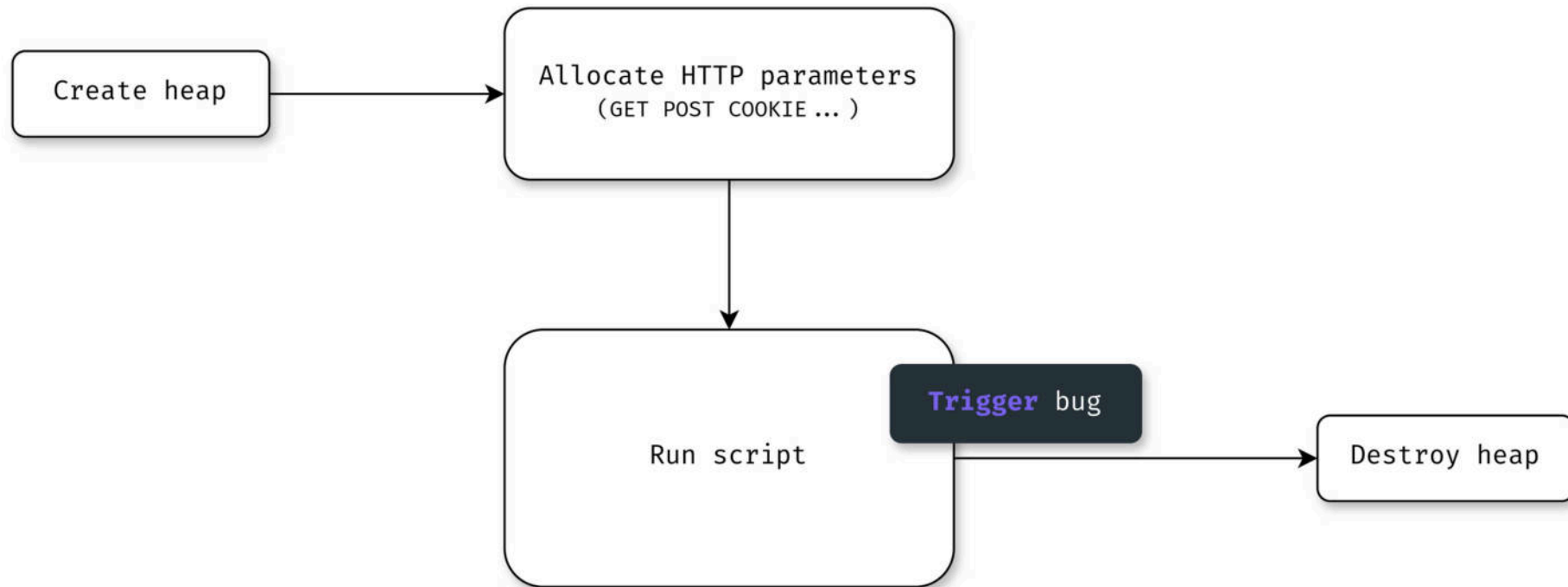
Request lifecycle

1 heap for 1 request
(bugs do not linger)



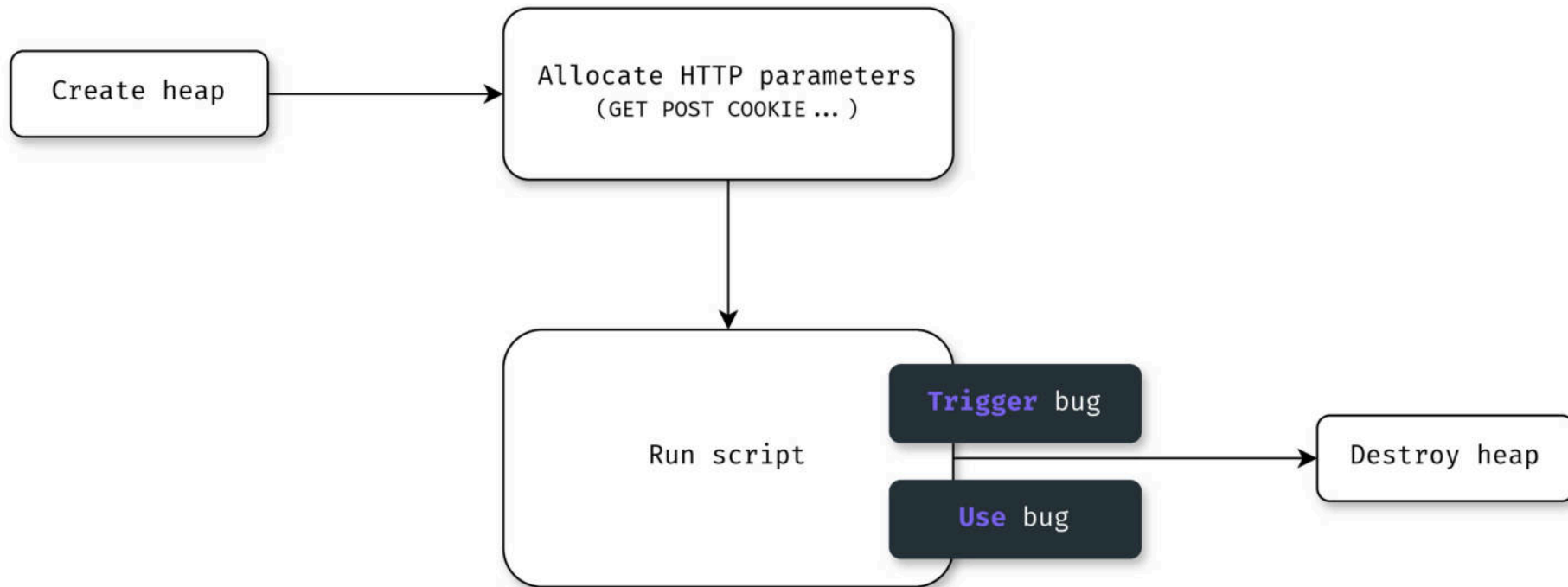
Request lifecycle

Triggering the **bug**
(finding a vulnerable code path)



Request lifecycle

Using the **bug**
(in the same request)



Handling of PHP filters

`php://filter/[filters]/resource=[resource]`

PHP processes a `php://filter`
in 2 steps

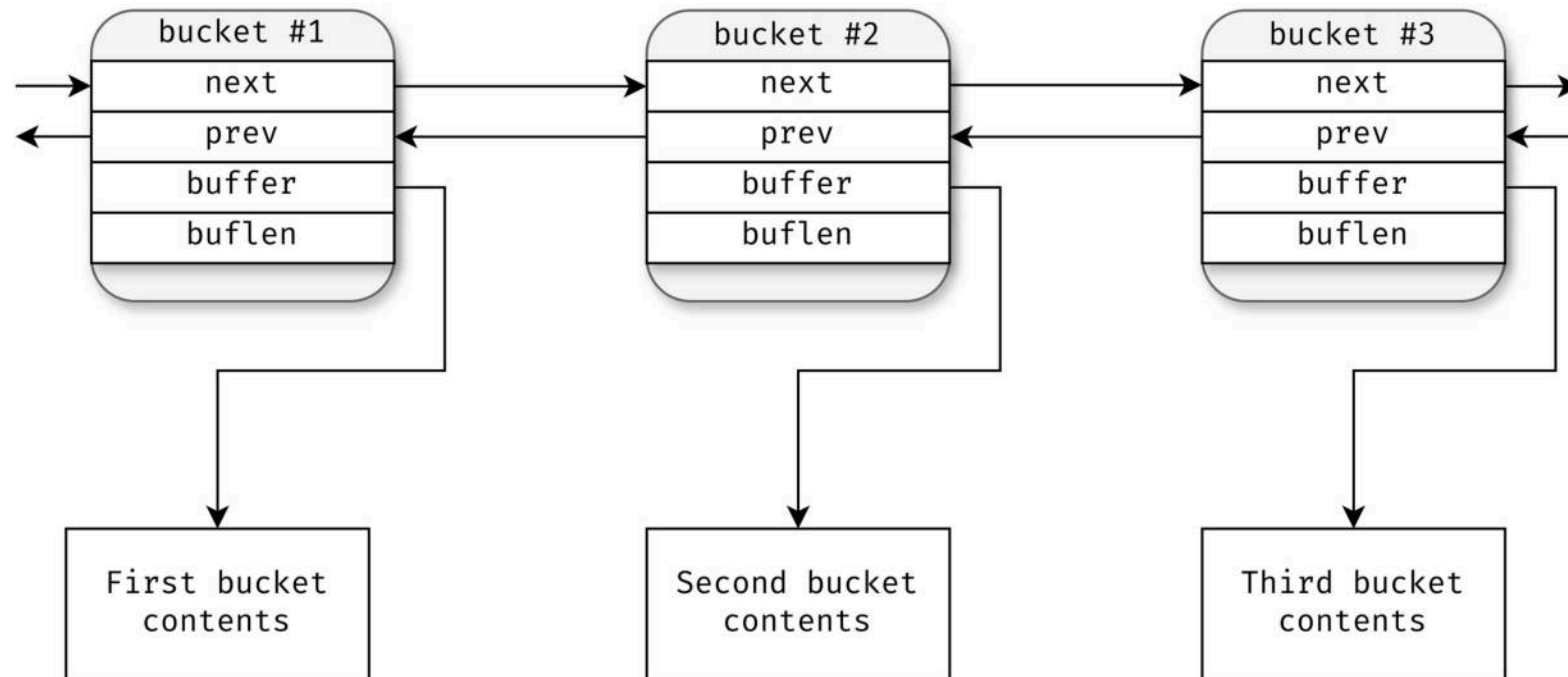
1. Read the resource as a stream
2. Apply filters



Handling of PHP filters

php://filter/[filters]/resource=[resource]

1. Fetching the stream



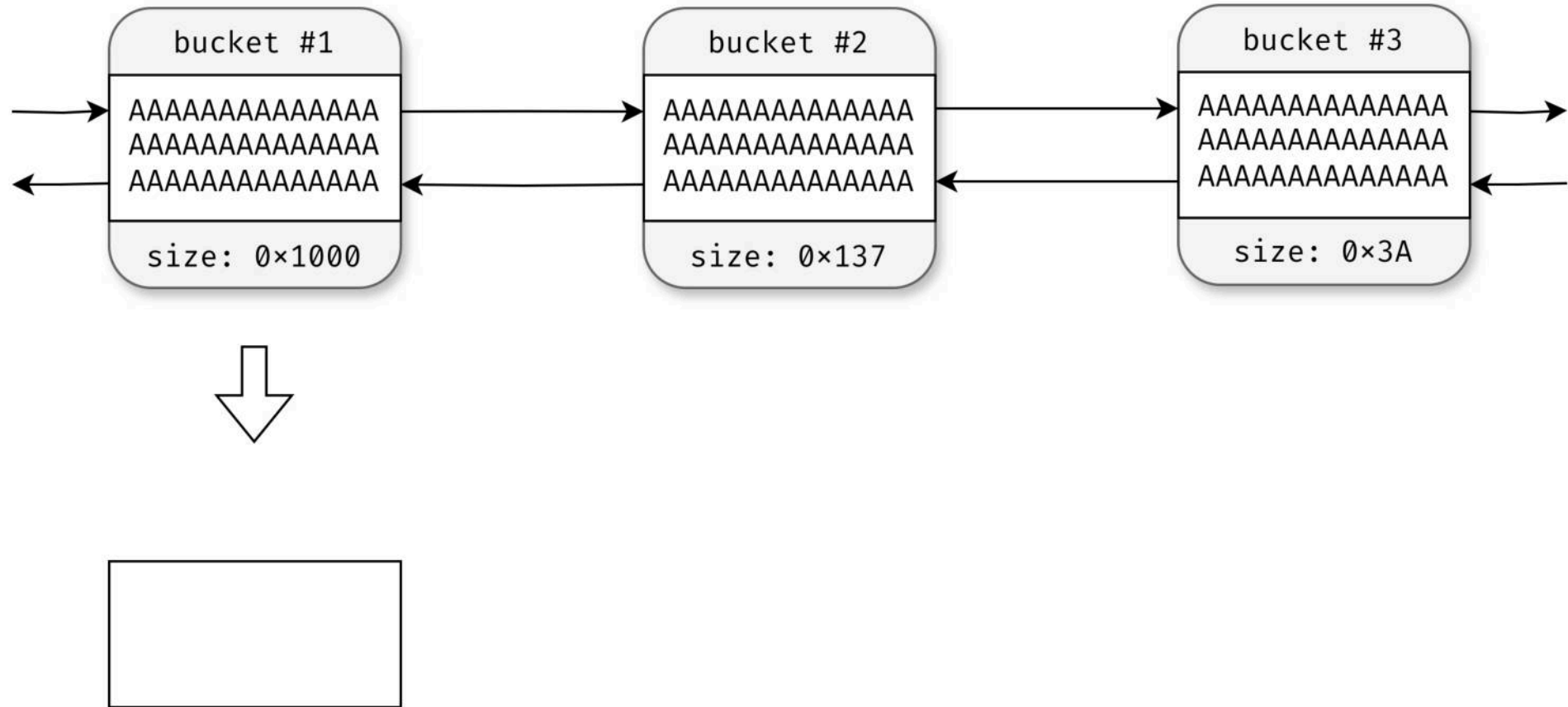
Handling of PHP filters

2. Applying filters



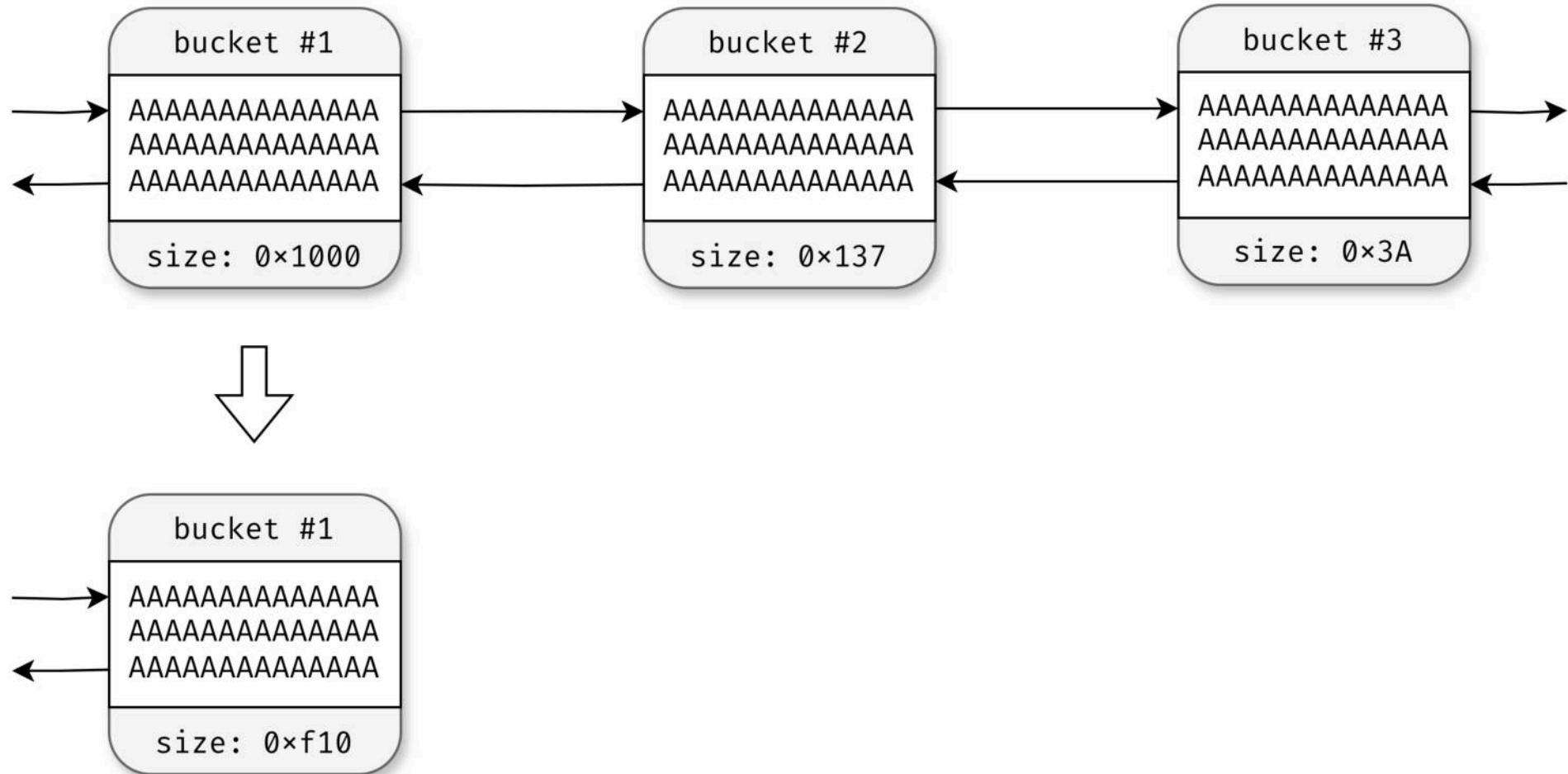
Handling of PHP filters

3. Applying filters



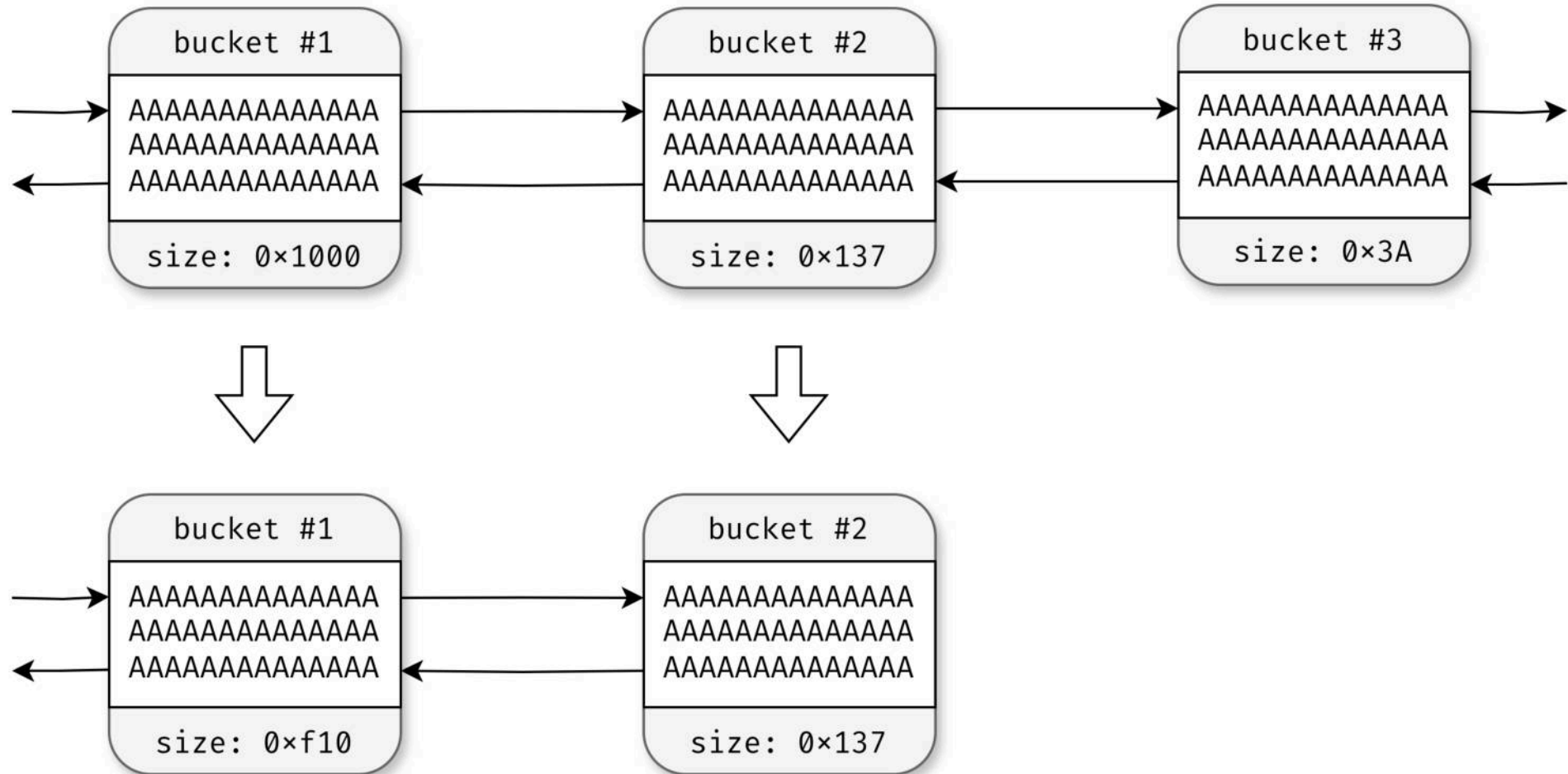
Handling of PHP filters

3. Applying filters



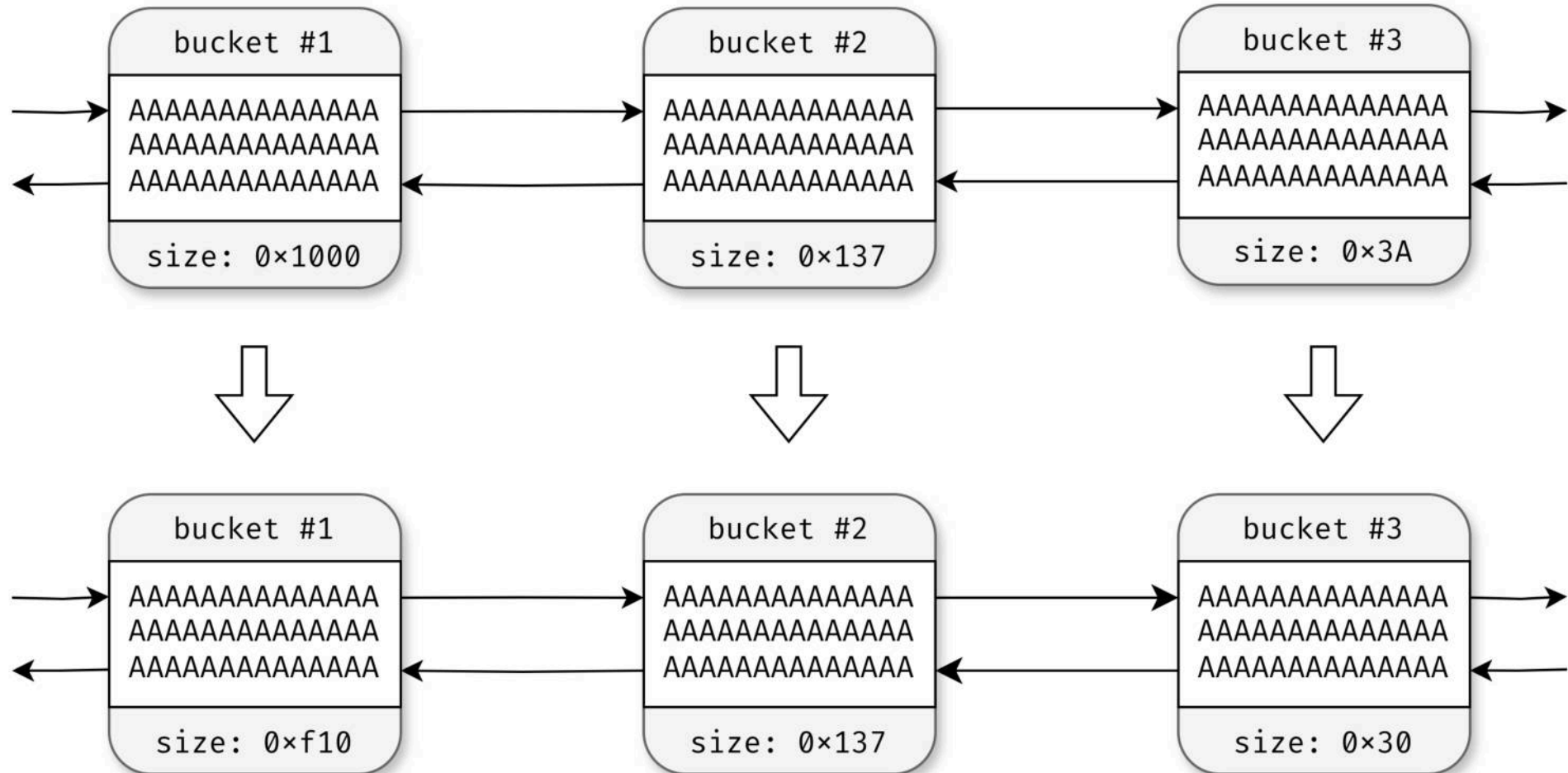
Handling of PHP filters

3. Applying filters



Handling of PHP filters

3. Applying filters



Building an exploit: file read

```
echo file_get_contents($_GET['file']);
```

Not too hard ?

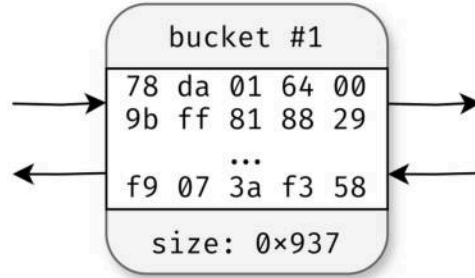
- Read binaries (php, libc)
- ASLR/PIE is **NOT** a problem
- Address of PHP's heap is **known**
- Arbitrary allocations/frees

But...

- **Unknown** environment:
 - Application:
 - Wordpress, Drupal, Symfony...
 - PHP version
 - Heap state
- Need a **reliable** exploit



Building an exploit: a **single** bucket



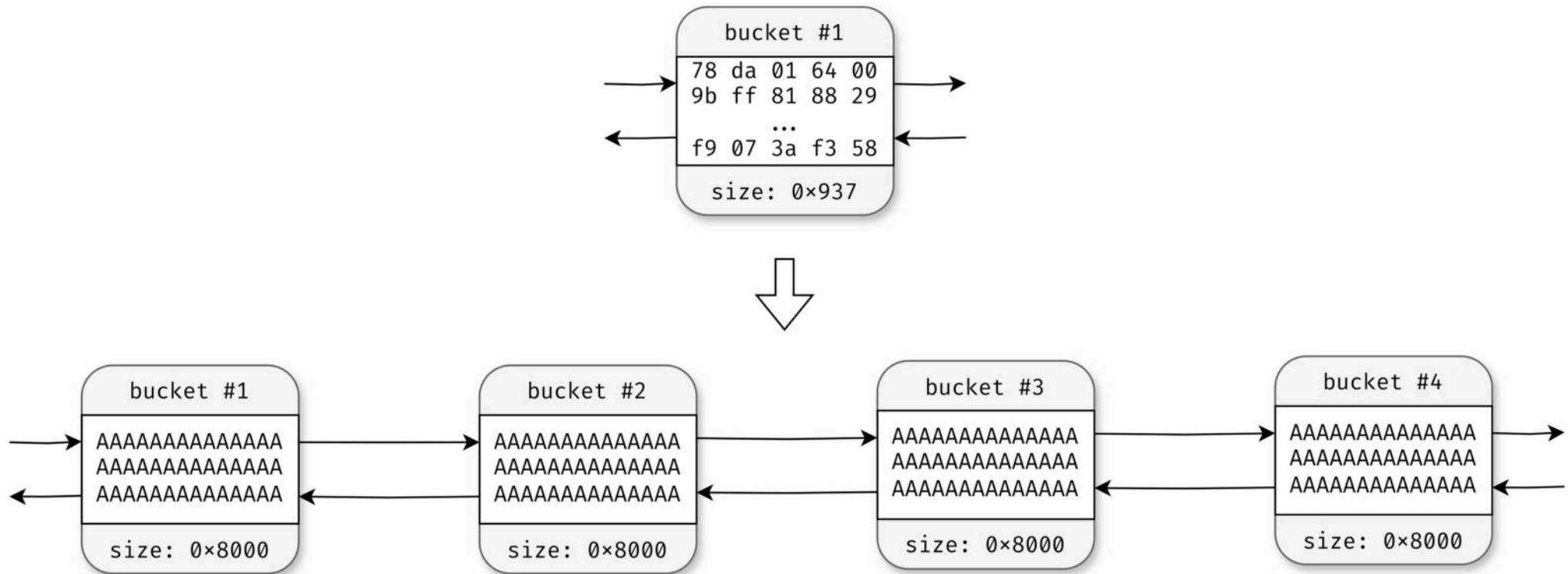
Streams are read in **one** bucket

Impractical for exploitation

- Cannot :
 - Spray
 - Pad
 - Use altered free list



Building an exploit: using [zlib.inflate](#) to create buckets



Building an exploit: unfitting chunk size

chunk size (as hex)

chunk data

```
8␣  
This is ␣  
11␣  
how the chunked e␣  
d␣  
ncoding works␣  
0␣
```

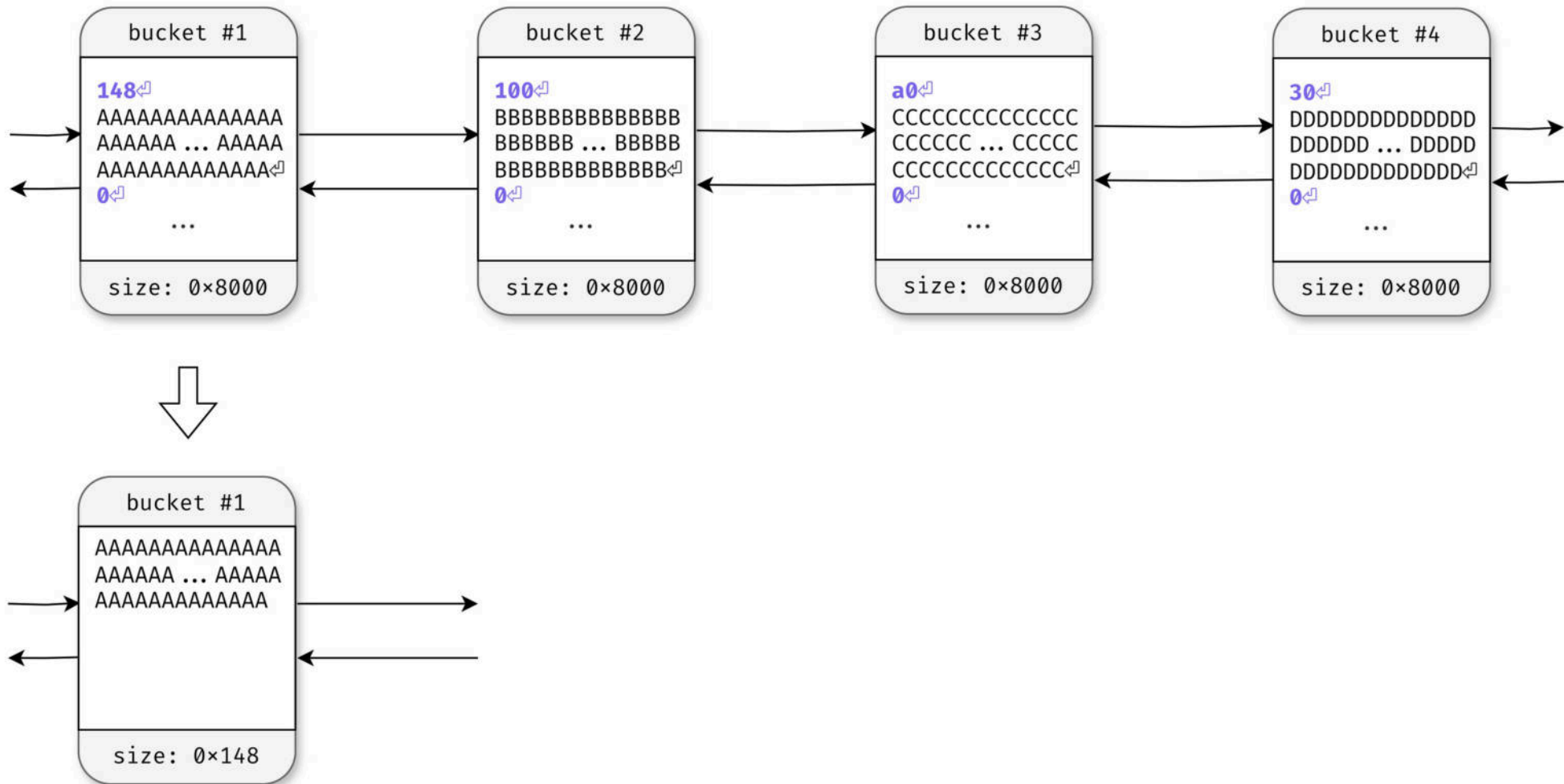
Resizing chunks to arbitrary sizes

*Using the undocumented **dechunk** filter*

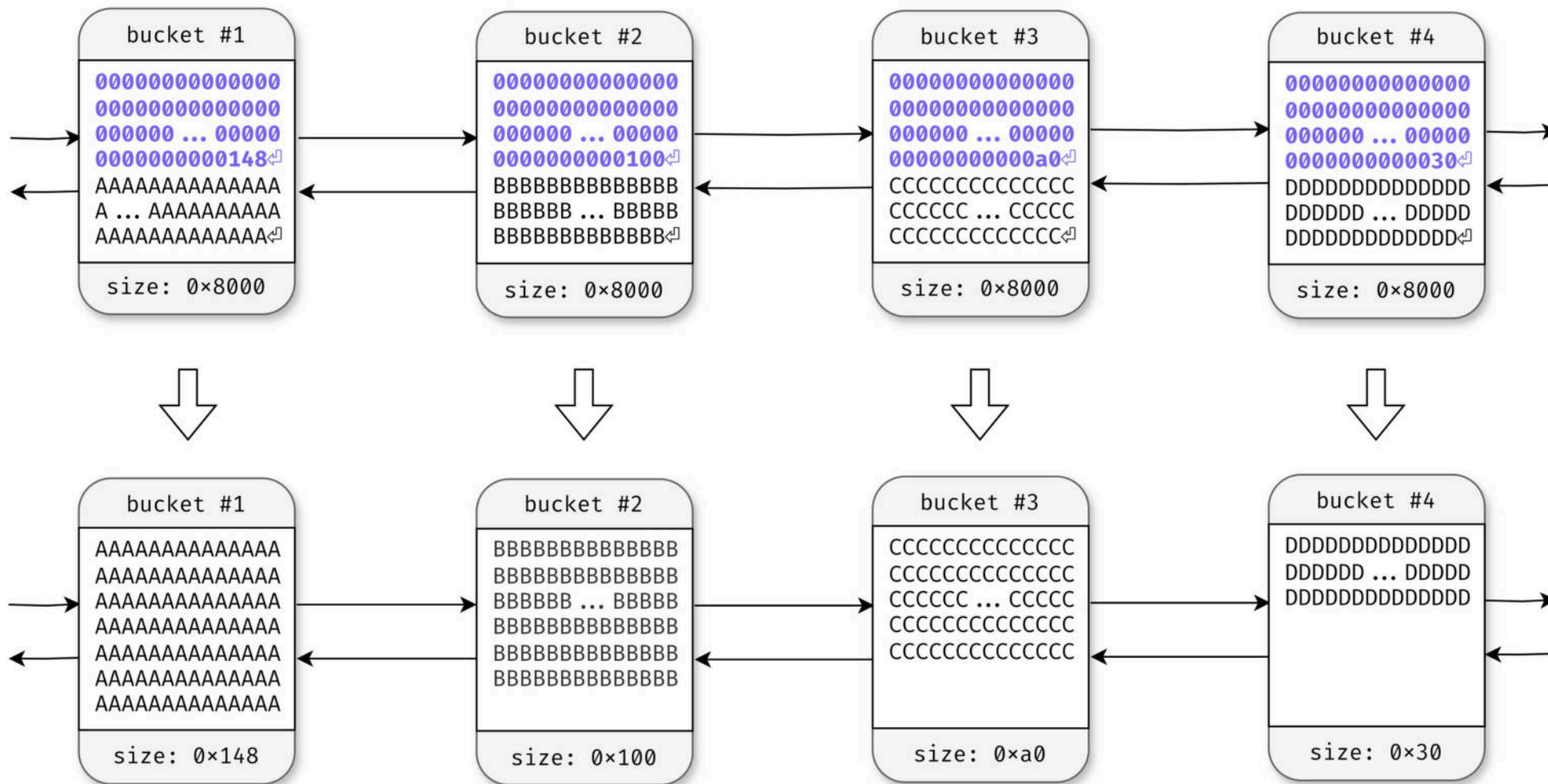
« Decodes a string which is HTTP-chunked encoded. »



Building an exploit: unfitting chunk size (0x8000)



Building an exploit: using `dechunk` to change chunk sizes



Building an exploit: arbitrary allocations



Chunk attack

- We attack chunks of size 0x100
- Chunks of other sizes do **not** matter
- Because they are not in the same page

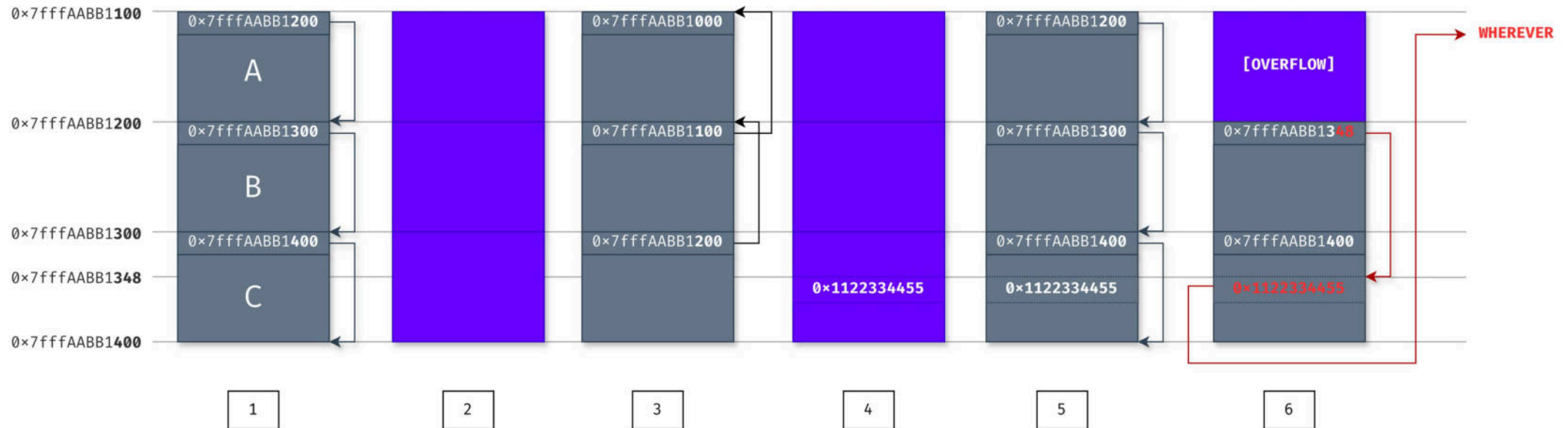
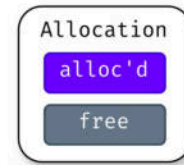
Dechunking buckets

- Use **dechunk** to make buckets **appear/disappear**
- Buckets **appear** when they are needed



Building an exploit: controlling a free list

1-byte overflow (48h)
on chunks of size 0x100
to overwrite a `→next` pointer



Building an exploit: endgame

```
php-8.3.2/Zend/zend_alloc.c

1 struct _zend_mm_heap {
2     // Whether to use custom allocation functions
3     int         use_custom_heap;
4
5     ...
6
7     // Free lists for each chunk size
8     zend_mm_free_slot *free_slot[ZEND_MM_BINS];
9
10    ...
11
12    // Custom allocation functions
13    struct {
14        void      *(*_malloc)(size_t);
15        void      (*_free)(void*);
16        void      *(*_realloc)(void*, size_t);
17    } custom_heap;
18
19    ...
20 };
```

Code execution

- 1-byte overflow (**0x48**) to alter a free list
- Allocate at arbitrary address
- Overwrite **zend_mm_heap**
 - At the top of the heap region
- Overwrite function pointers
 - **free** == **system**
- Call **free(command)**



Building an exploit

CNEXT performance

- ✓ Works against **any** target
 - From PHP 7+ to current (2015 to 2024)
 - Any PHP application
- ✓ **100%** reliable
 - No crash
- ✓ Payload **< 1000** bytes
 - Suitable for GET parameters
- ✓ Self-contained exploit
 - No additional HTTP parameters

File read → **RCE**



Demo: Magento XXE to RCE

CosmicSting

Magento XXE

(CVE-2024-34102)

June, 2024

IMPACT

Without cnext

Read files

Create API keys

With cnext

Remote

Code

Execution





Exploiting direct iconv() calls

Another sink: `iconv()` calls

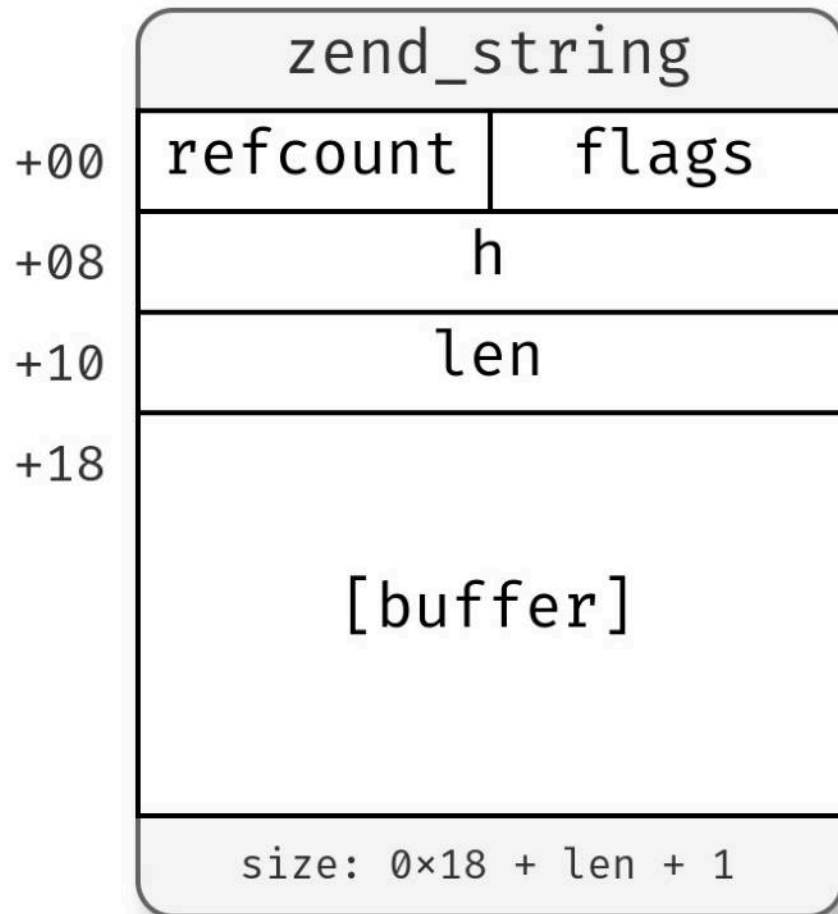
```
iconv(string $from_encoding, string $to_encoding, string $string): string
```

```
    iconv_substr( ... ) iconv_mime_decode( ... )  
    iconv_mime_decode_headers( ... ) iconv_strlen( ... )  
    iconv_strpos( ... ) iconv_strrpos( ... )
```

- Direct calls to `iconv*()` are vulnerable
- Exploitation is completely different



Exploit strategy: getting a leak



Info leak

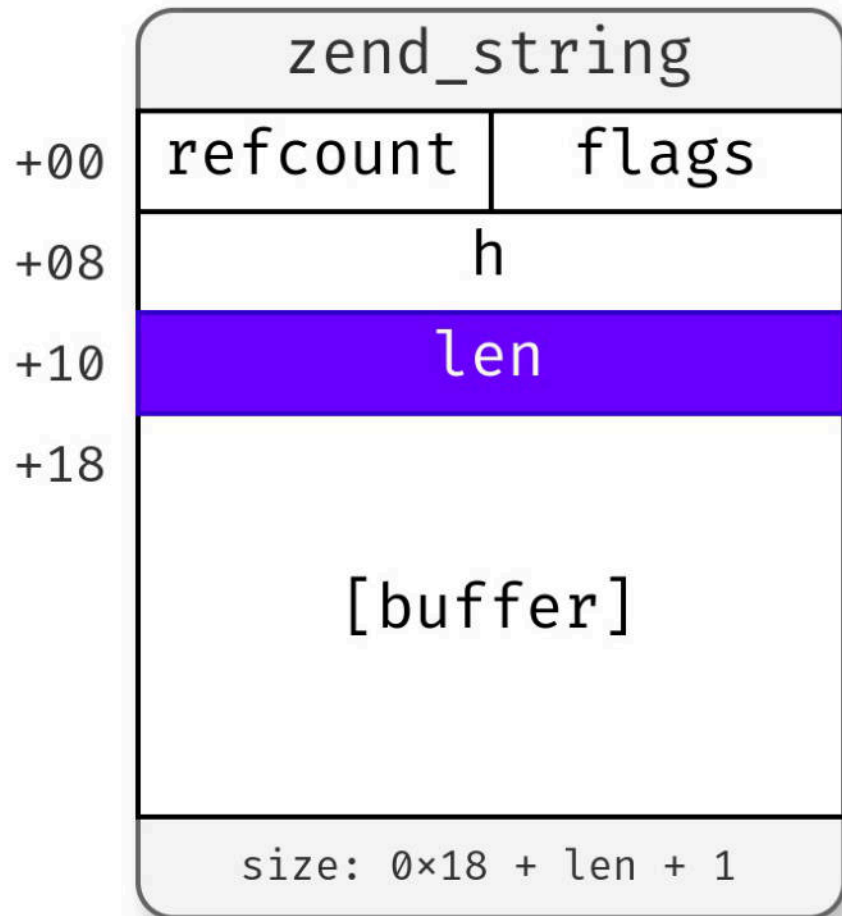
- Overwrite → **len** of a **zend_string**
- *Before* it is displayed

Bad bug

- 1 to 3 bytes overflow...
- Can only overwrite **refcount**
- Need to **alter free list**



Exploit strategy: getting a leak



Steps

- Corrupt free list
 - Two chunks overlap
- Allocate on top of string
 - Overwrite → `len`
- String gets displayed



Exploit strategy: reliance on the code

We rely on the code

After we send HTTP parameters, we **cannot** interact with the script anymore

Need to use « **code gadgets** » to perform actions



Exploit strategy: reliance on the code

We rely on the code

```
1 <?php
2
3 $text = iconv('UTF-8', $_POST['charset'], $_POST['text']);
4
5 # end of script
6
```

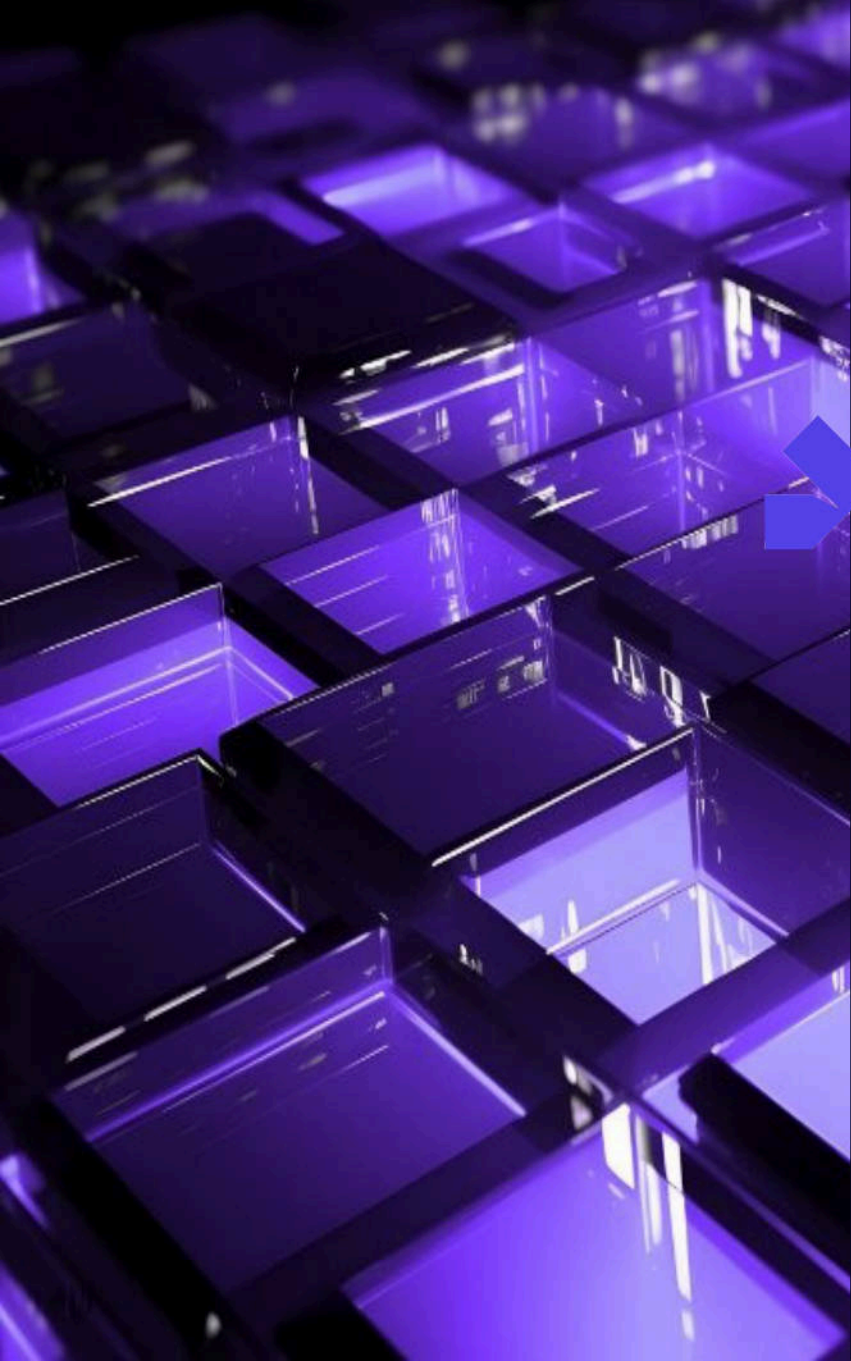


Exploit strategy: reliance on the code

We rely on the code

```
1 <?php
2
3 $text = iconv('UTF-8', $_POST['charset'], $_POST['text']);
4
5 $lines = explode("\n", $text); # ← a way to use the bug
6
```





Real-world exploit

Attacking Roundcube



« Free and open source webmail software for the masses, written in PHP. »



Attacking Roundcube: codewise

When creating an email...

```
program/include/rcmail_sendmail.php

1  $charset = rcube_utils::get_input_string('_charset', INPUT_POST);
2
3  $mailto = $this->email_input_format(
4      rcube_utils::get_input_string('_to', INPUT_POST, $charset)
5  );
6  $mailcc = $this->email_input_format(
7      rcube_utils::get_input_string('_cc', INPUT_POST, $charset)
8  );
9  $mailbcc = $this->email_input_format(
10     rcube_utils::get_input_string('_bcc', INPUT_POST, $charset)
11 );
12
13 if (!empty($this->invalid_email)) {
14     display_error('emailformaterror', 'error', [
15         'email' => $this->invalid_email
16     ]);
17 }
```



Attacking Roundcube: memory leak strategy

Simple strategy

1. Set `_to` to an invalid email $\Rightarrow$ `invalid_email` gets set
2. Use `_cc` to trigger bug $\Rightarrow$ free list altered
3. Use `_bcc` to allocate on top of `invalid_email`, altering its size
4. Error is displayed $\Rightarrow$ memory leak



Attacking Roundcube: memory leak failure

A slight oversight

error message is **JSON**-encoded

[illegible]

Attacking Roundcube: **working** memory leak strategy

```
include/rcmail_output_html.php

1 function get_js_commands(&$framed = null)
2 {
3     $unlock = isset($_REQUEST['_unlock']) ? preg_replace('/[^a-z0-9]/i', '', $_REQUEST['_unlock']) : 0;
4
5     if ($this->framed) {
6         $top_commands[] = ['iframe_loaded', $unlock];
7     }
8     else if ($unlock) {
9         $top_commands[] = ['hide_message', $unlock];
10    }
11
12    $commands = array_merge($top_commands, $this->js_commands);
13
14    foreach ($commands as $i => $args) {
15        $method = array_shift($args);
16
17        foreach ($args as $i => $arg) {
18            $args[$i] = self::json_serialize($arg, $this->devel_mode);
19        }
20
21        $method = self::JS_OBJECT_NAME . '.' . $method;
22
23        $out .= sprintf("%s(%s);\n", $method, implode(',', $args));
24    }
25
26    $framed = $parent_prefix && $parent_commands = count($commands);
27
28    // make the output more compact if all commands go to parent window
29    if ($framed) {
30        $out = "if (window.parent && parent." . self::JS_OBJECT_NAME . ") {\n"
31            . str_replace($parent_prefix, "\t\tparent.", $out)
32            . "}\n";
33    }
34
35    return $out;
36 }
```

Everything is useful
(de)allocations are everywhere

(de)allocations

Made implode() of the
second iteration of the
loop overlap with \$out



Attacking Roundcube: Leak to RCE

RCE is just around the corner

- Data only attack:
 - overwrite **roles**, **serialized** data, ...
- Binary attack:
 - overwrite **zend_array** → **pDestructor**

Visible part of the iceberg

- Need proper heap setup
- Variable scope is hell
- Very **specific** to server + application
- Generally a huge **pain** to pull off



Attacking Roundcube: DEMO

DEMO





Conclusion

Conclusion: a bad bug... but with some uses

Targeting PHP

- New attack vector: file read to RCE
- Application specific attacks: Roundcube RCE

Other targets

- Other applications may be vulnerable
- Discovered as time go by?



Conclusion: timeline

Timeline (2024)

- Last year Crash found
- February Started working on bug
- March 26th Bug reported to glibc security team
 - They did an **amazing** job!
- April 04th Bug reported to linux distros
- April 17th Bug released as CVE-2024-2961



Conclusion: resources

Blogposts and exploits

- ambionics.io/blog
- github.com/ambionics/cnext-exploits
- twitter.com/cfreal_



PARIS LYON TOULOUSE

+33 (0)1 40 17 91 28
contact@lexfo.fr

ambionics.io/blog
blog.lexfo.fr

@LexfoSecurite
@ambionics

Charles FOL
@cfreal_

Lexfo 



What ifs

What if: **blind** file read

```
echo file_get_contents($_GET['file']);
```

What if there is no **output**?

- No /proc/self/maps
- No access to binaries



What if: no `zlib.inflate`

What if there is no `zlib.inflate`?*

- Attack with a `single` bucket
- Cannot pad heap
- Cannot setup the free list
- Cannot even `USE` the altered free list

* Very unlikely



Blind, single bucket exploit: DEMO

DEMO

Blind, single-bucket exploit



PARIS LYON TOULOUSE

+33 (0)1 40 17 91 28
contact@lexfo.fr

ambionics.io/blog
blog.lexfo.fr

@LexfoSecurite
@ambionics

Charles FOL
@cfreal_

Lexfo 